

Monster Lab

The Tuesday story: build Git, one problem at a time.

Each lab builds one monster. Every button runs real Git.

Feel the problem. Invent the fix. See what Git does behind the door.

STEP 0

Everyone, one monster

PROBLEM

Everyone edits one monster at once. "Who changed the legs?" Nobody knows.

IDEA

Save every version, with a name on it.

GIT

None yet. This is life without it.

Monster Lab • Lab 1 • Ana • Step 0 • Everyone, one monster

STEP 0

Everyone, one monster

Your lab shares one monster.
Change any part, any time. Go!

Your lab's monster



FACE
frog

change ›



BODY
box

change ›



LEGS
wheels

change ›

STEP 1

Save every version

PROBLEM

You could not get an old monster back.

IDEA

Never erase. Every save is a new card: the whole monster, your name, the card before.

GIT

git commit. The card's ID is a hash of all that. Following the arrows back is git log.

Monster Lab • Lab 1 • Ana • Step 1 • Save every version

STEP 1

Save every version

The chaos monster is gone. From now on, every save makes a card. One at a time: change one part, then press **Save card**. Everyone saves at least once.

✓ Everyone saved a card (3/3)

Bonus • Click any card. Press **Show what Git stored**. Find the parent, the author and the message.

Discuss: Why do arrows point back, never forward?

» Behind the door • what Git did

Shared draft

FACE
frogchange ›

BODY
coatchange ›

LEGS
duckchange ›

Save card
git commitNEW

All saved

Your lab's cardsgit log

235b510
Monster Lab
Start

7fab12d
Ana
FACE: smiley → frog

aeac253
Raj
BODY: box → coat

79233fb
Mei
LEGS: sticks → duck

← points to the card before

STEP 2

Try two ideas at once

PROBLEM

Two risky ideas could wreck the good monster.

IDEA

Each pair gets its own sticky note. Saving moves only theirs.

GIT

git branch: a tiny file holding one card's ID. Your pin (HEAD) is the note you are on.

Monster Lab

Lab 1

Ana

Step 2

Try two ideas at once

STEP 2

Try two ideas at once

Your lab already has one sticky note: main. Each pair makes its own note for its idea. Build on it and save.

YOUR MISSION

One of you: press **New sticky note**, keep the name **cat-robot**. The other: wait for it, then **Switch to cat-robot**. Then FACE → 🐱, BODY → 🤖. **Save card**.
You're in Pair A with Mei ([change](#))

✓ cat-robot has 🐱 + 🤖

✓ superhero has 🦸 + 🦋

Bonus · Switch to main and back. Watch the draft change. Look, don't edit.

Behind the door

what Git did

Shared draft

You're on: cat-robot

FACE

cat

change >

BODY

robot

change >

LEGS

duck

change >

All saved

New sticky note

git switch -c

NEW

Switch to

main

git switch

NEW

Save card

git commit

Your lab's cards

git log

235b510

7fab12d

aeac253

79233fb

c7e6170

4c2f031

← points to the card before

sticky note

YOU your pin: the note you're on

STEP 3

Make one monster from both

PROBLEM

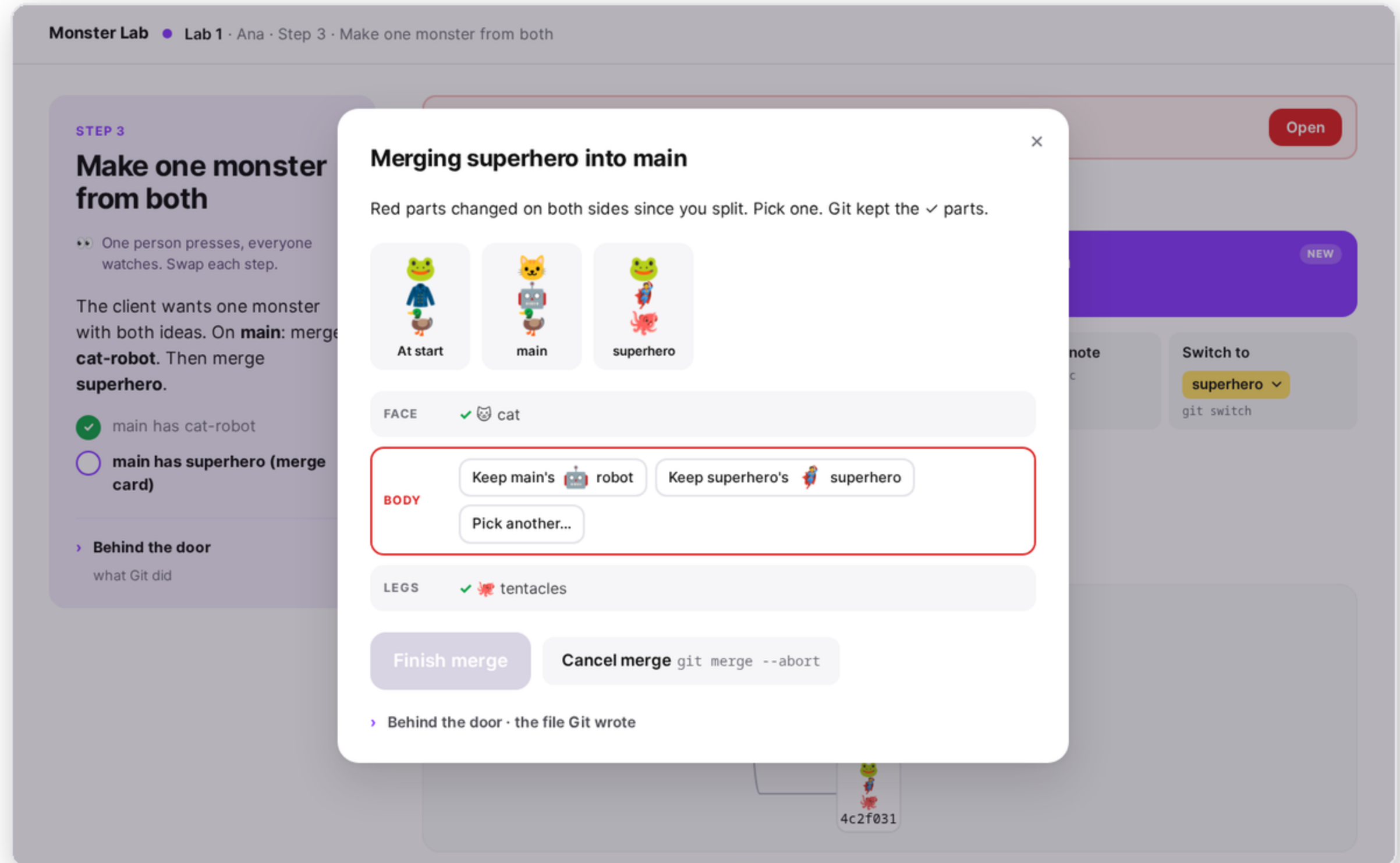
The client wants one monster with both ideas.

IDEA

Compare each part with where both ideas started. Changed on one side: keep it.

GIT

git merge. FACE and LEGS combine alone. BODY changed on both sides: a person decides.



STEP 4

Meet the Wall

PROBLEM

Each lab's cards live only in that lab.

IDEA

One shared copy for the class, like GitHub. Every lab gets a full copy.

GIT

git clone. The same card has the same ID on every laptop: the ID is computed from it.

STEP 4

Meet the Wall

The Wall is the class's shared copy, like GitHub. It got your lab's cards, and every lab got a copy. Is your newest card's ID the same as the Wall's?

- Behind the door • what Git did

Shared draft

You're on: main

FACE
cat

BODY
shell

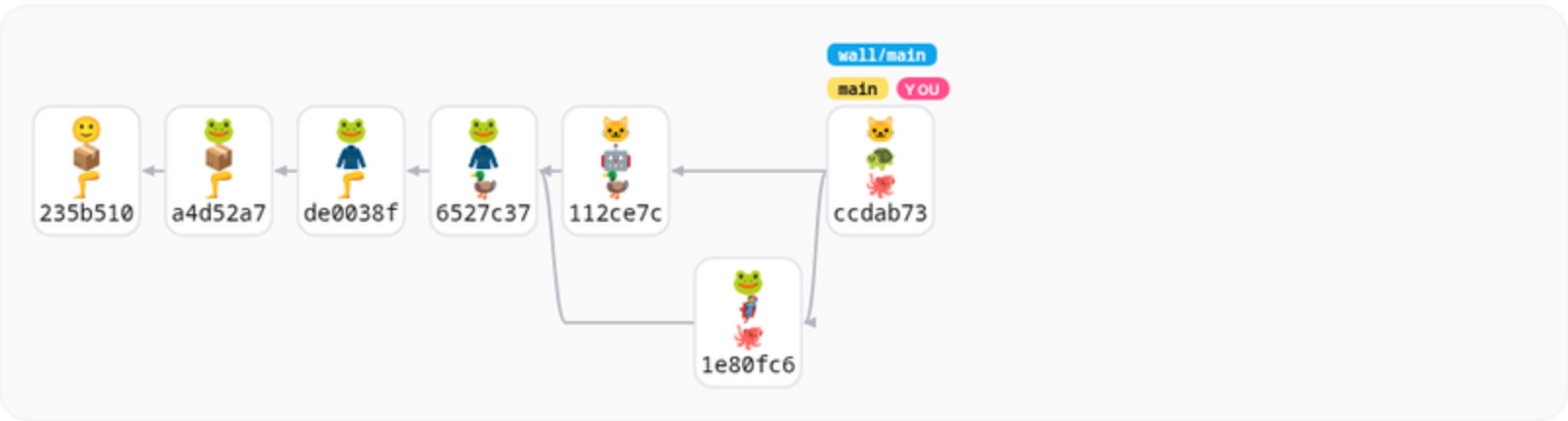
LEGS
tentacles

Save card
git commit

New sticky note
git switch -c

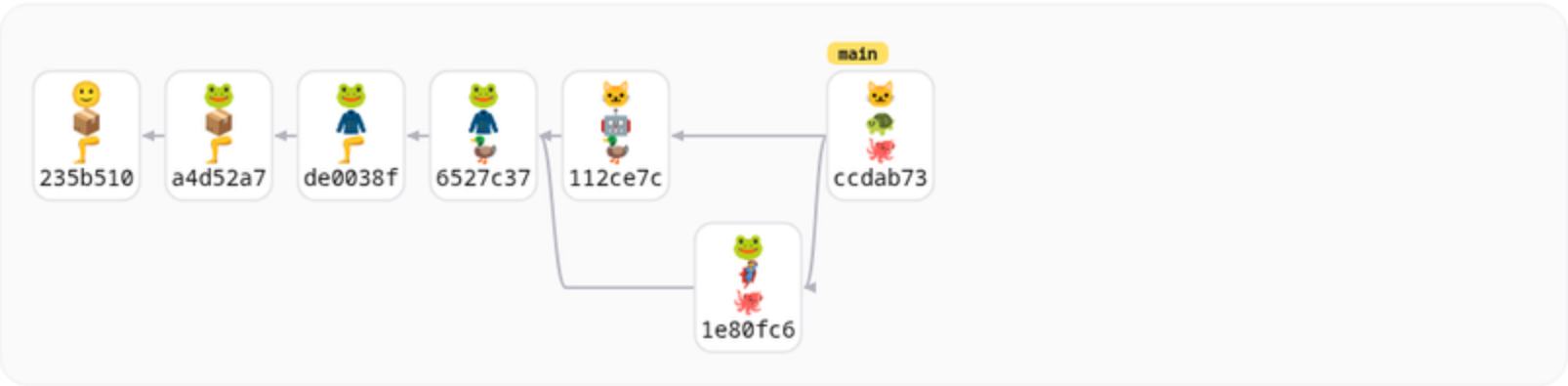
Look only in this step. You change main in Step 5.

Your lab's cards git log



← points to the card before sticky note YOU your pin: the note you're on the Wall, last time you checked

• The Wall the class's shared copy



STEP 5

Put your monster on the Wall

PROBLEM

The Wall refused your card.

IDEA

The Wall never drops a card. Get its new cards, combine, then send.

GIT

git push only moves the Wall forward. git pull = git fetch + git merge.

Monster Lab • Lab 2 • Priya • Step 5 • Put your monster on the Wall

Refused? Press **Get & combine**, then send again.

YOUR MISSION

LEGS → 🦿. Change nothing else.

- ☐ Your lab's change is on the Wall
- ☐ Your lab's cards match the Wall (Get & combine)

Behind the door

what Git did

git push is refused unless you already have the Wall's newest card.

git pull = git fetch + git merge .

Last: **Priya** • Get & combine → merge card c72bc85

git fetch wall

git merge wall/main

Git fetched the Wall's cards, then made merge card c72bc85 with two parents.

Show the low-level steps Git ran

```
$ git fetch wall
From ../.. /wall
  ccdab73..aad7037  main    ->
  wall/main
```

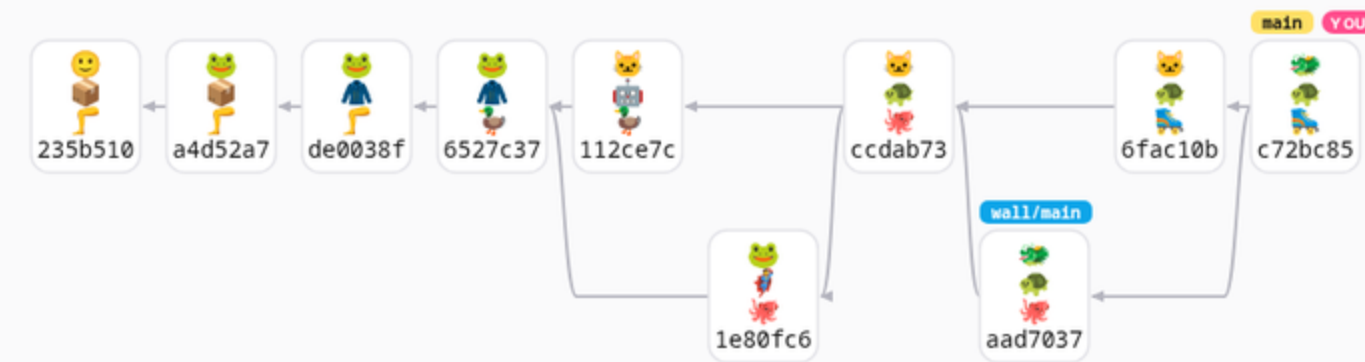


skates

change ›

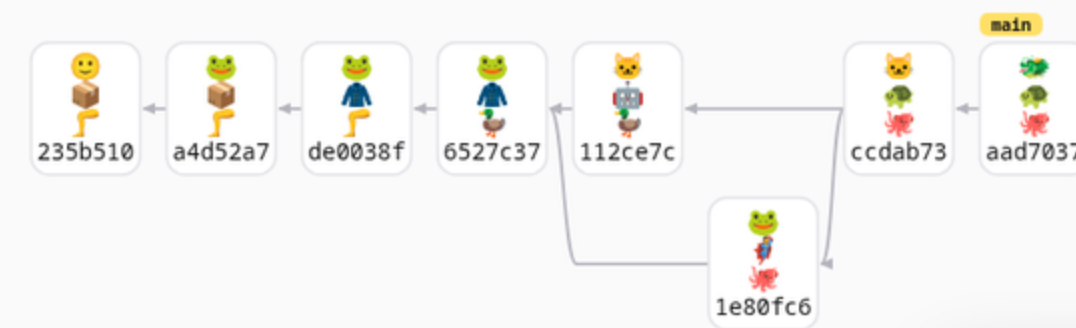
All saved

Your lab's cards git log



← points to the card before 🟡 sticky note **YOU** your pin: the note you're on 🟢 the Wall, last time you checked

• The Wall the class's shared copy



Got the Wall's cards and combined them. Now press Send to Wall.

STEP 6

Oops: undo a shared mistake

PROBLEM

A mustache card reached the Wall, and every lab has it.

IDEA

Don't rip out a shared card. Add a card that undoes it.

GIT

git revert adds a fix card. git reset moves your note back. git reflog remembers every move.

Monster Lab • Lab 1 • Ana • Step 6 • Oops: undo

STEP 6

Oops: undo a shared mistake

↔ One person presses, everyone watches. Swap each step.

A mustache card reached the Wall. Get it, then remove it without breaking anyone's copy.

YOUR MISSION

Press **Get & combine**. In your lab's cards, click the 🤪 card. Press **Undo this card**, then **Send to Wall**.

✔ You got the mustache card

○ No mustache on the Wall or in your lab (Undo, then Send)

Behind the door

what Git did

🤪

🍩

🐾

CARD ID

b6ae286

b6ae28682d4115f5cc9b79a180d164bbd7251a8e

Tiny style fix

Made by The Intern · a moment ago

Parent **5e8482a**

▼ Show what Git stored

```
$ git cat-file -p b6ae28682d4115f5cc9b79a180d164bbd7251a8e

tree 739fc456d9ebb0e1f55ae0c500fba3da5e89da09
↳ the snapshot of the files (monster.txt), by its ID
parent 5e8482ae4d5818a1f1c7a4303a44feec907e2e73
↳ the card before
author The Intern <intern@monster.lab> 1791161363 +0000
↳ who made it, and when (seconds since 1970)
committer The Intern <intern@monster.lab> 1791161363 +0000
↳ who saved it (here, the same)

Tiny style fix
↳ the message

$ git cat-file -p
b6ae28682d4115f5cc9b79a180d164bbd7251a8e:monster.txt

face: mustache
---
body: donut
---
legs: paws
```

Undo this card

Move my note back here

NEW

move your note back.

New sticky note

git switch -c

Send to Wall

git push

graph LR
 671a[671a] --> d41d49e[d41d49e]
 d41d49e --> 5e8482a[5e8482a]
 5e8482a --> b6ae286[b6ae286]
 b6ae286 --> dbaf438[dbaf438]
 dbaf438 --> d41d49e
 subgraph Wall
 5e8482a
 b6ae286
 end
 subgraph Lab
 671a
 d41d49e
 dbaf438
 end
 style 5e8482a fill:#f9f9f9,stroke:#333,stroke-width:1px
 style b6ae286 fill:#f9f9f9,stroke:#333,stroke-width:1px
 style 671a fill:#f9f9f9,stroke:#333,stroke-width:1px
 style d41d49e fill:#f9f9f9,stroke:#333,stroke-width:1px
 style dbaf438 fill:#f9f9f9,stroke:#333,stroke-width:1px

← points to the card before

🍩 sticky note

YOU your pin: the note you're on

the Wall, last time you checked

only in the diary

STEP 7

The boss wants it clean

PROBLEM

"Who added the tentacles?" Before the clean-up: Tom, Lab 2. After: not found.

IDEA

Rewriting history makes new cards and forgets the old ones.

GIT

Squash = one new card, new ID. git push --force makes the Wall forget. The paper: flat history is data loss.

Step 7 · The boss wants it clean

1:56

The Tuesday paper: flat history is data loss.

THE WALL



BEFORE THE CLEAN-UP · WHO FIRST ADDED LEGS 🦿?

Wall: Tom (Lab 2), 1e80fc6

Lab 1: Tom (Lab 2), 1e80fc6
Lab 2: Tom (Lab 2), 1e80fc6
Lab 3: Tom (Lab 2), 1e80fc6

AFTER THE CLEAN-UP · WHO FIRST ADDED LEGS 🦿?

Wall: not found

Lab 1: not found
Lab 2: Tom (Lab 2), 1e80fc6
Lab 3: Tom (Lab 2), 1e80fc6

• Lab 1

✓ The Wall has one clean card

• Lab 2

✓ The Wall has one clean card

• Lab 3

✓ The Wall has one clean card

STEP 8

What you built

Cards never change.

Sticky notes move.

The Wall copies cards.
That's Git.

What you built

Cards never change. Sticky notes move. The Wall
copies cards. That's Git.

Lab	Cards saved	Merges	Conflicts solved	Pushes	Refused pushes	Reverts	Git ideas used
● Lab 1	7	1	1	3	0	1	Save 7 Branch 2 Fast-forward 4 Merge Conflict solved Push 3 Refused push Pull 5 Revert Reset Diary Force push
● Lab 2	6	2	1	1	2	0	Save 6 Branch 2 Fast-forward 7 Merge 2 Conflict solved Push Refused push 2 Pull 7 Revert Reset Diary Force push
● Lab 3	7	3	1	2	4	0	Save 7 Branch 2 Fast-forward Merge 3 Conflict solved Push 2 Refused push 4 Pull 2 Revert Reset Diary Force push

TEACHERS

What you see while it runs

SCRIPT

Each step's problem, the one question, and the answer you hope to hear.

LIVE

Every lab's cards, goals and alerts like "Refused twice in a row".

FEED

Who did what, with the real Git command.

Monster Lab · Teacher

Step 5 · Put your monster on the Wall

0:10 / 5:00 · 42 min ahead Restart timer



http://127.0.0.1:3102/

9 people · 9 online

Labs 3

NOTES Watch for: the first lab gets in. The others are refused, then combine with no red. "Refused twice"? Check they pressed Get & combine.
Board: "5. Send (push) only moves the Wall forward. Behind? Get & combine (pull) first."
The Wall refused your card.
PAUSE · ASK Why did the Wall refuse your card instead of adding it?
Ask on the projector
HOPE TO HEAR It would drop the first lab's card. The Wall only moves forward.
IDEA The Wall never drops a card. Combine its new cards first.
HOW GIT DOES IT git push is refused unless you already have the Wall's newest card. git pull = git fetch + git merge.
Each card's trip to the Wall is an integration path. How long the trip took is code velocity. Microsoft had to rebuild that metric for Git.

NEXT · STEP 6 · OOPS: UNDO A SHARED MISTAKE

PROBLEM A bad card is on the Wall, and every lab has it.
ASK FIRST After Sabotage and Get & combine: "How do you get rid of it?"
IDEA Don't rip out a shared card. Add a card that undoes it.

Prev

Start Step 6

Adds Undo, Move back and the Safety diary. Press Sabotage first, on Step 5. If you didn't, Next does.

Sabotage

Audit: who added

Empty the Wall's bin git gr

Reset session

Open projector

Lab 1

3 people

- ✓ Your lab's change is on the Wall
- ✓ Your lab's cards match the Wall (Get & combine)

Ana A main
Raj B main
Mei A main



Ana · Get & combine → fast-forward · just now

Save 7 Branch 2 Fast-forward 3 Merge
Conflict solved Push 2 Refused push Pull 3

Lab 2

3 people

- ✓ Your lab's change is on the Wall
- ✓ Your lab's cards match the Wall (Get & combine)

Priya A main
Tom B main
Lea A main



Priya · Get & combine → fast-forward · just now

Save 6 Branch 2 Fast-forward 4 Merge 2
Conflict solved Push Refused push Pull 4

Lab 3

3 people

- ✓ Your lab's change is on the Wall
- ✓ Your lab's cards match the Wall (Get & combine)

Sam A main
Kim B main
Ola A main



Sam · Send to Wall → sent 5e8482a · just now

Save 7 Branch 2 Fast-forward Merge 3
Conflict solved Push 2 Refused push 4 Pull 2

The Wall wall.git



Feed

12:49 AM Lab 2 · Priya — Get & combine → fast-forward
git fetch wall
git merge wall/main
12:49 AM Lab 1 · Ana — Get & combine → fast-forward
git fetch wall
git merge wall/main
12:49 AM Lab 3 · Sam — Send to Wall → sent 5e8482a
git push
12:49 AM Lab 3 · Sam — Get & combine → merge card 5e8482a
git fetch wall
git merge wall/main
12:49 AM Lab 2 · Priya — Get & combine → fast-forward
git fetch wall
git merge wall/main
12:49 AM Lab 1 · Ana — Get & combine → nothing new
git fetch wall
git merge wall/main
12:49 AM Lab 3 · Kim — Send to Wall → refused (fetch first)
git push
12:49 AM Lab 3 · Sam — Send to Wall → refused (fetch first)
git push
12:49 AM Lab 3 · Ola — Send to Wall → refused (fetch first)
git push